

DevTrace — Distributed Developer Observability Engine



DevTrace v2 is a high-performance, developer-centric observability platform designed to capture, analyze, replay, and introspect API traffic in real time.

Unlike traditional logging tools, DevTrace operates as an **inline programmable proxy layer**, enabling developers to debug systems with production-grade fidelity.

Key Features

-  **Real-Time Interception:** Capture request-response cycles with zero-copy instrumentation.
-  **Replay Engine:** Resend captured requests directly from the CLI to simulate and debug edge cases.
-  **Plugin Architecture:** Extend the processing pipeline with custom Rust-based analyzers.
-  **Live Dashboard:** A modern Next.js interface for real-time traffic visualization and latency heatmaps.
-  **Time-Travel Debugging:** Powered by Immutable Logs (Event Sourcing) for deterministic troubleshooting.

Architecture

DevTrace follows a **CQRS (Command Query Responsibility Segregation)** pattern, splitting the high-throughput write path (capture) from the analytical read path (visualization).

```
flowchart TD
    Client["Client (Browser / Postman)"] --> Proxy["DevTrace Proxy (Rust)"]

    subgraph "Ingestion Layer (Rust)"
        Proxy --> Capture["Capture Engine"]
        Proxy --> Plugin["Plugin Engine"]
        Proxy --> Stream["Streaming Engine"]
    end

    subgraph "Data & Analysis"
        Plugin --> EventBus["Event Bus (Kafka / NATS / Tokio)"]
        EventBus --> EventStore["Event Store (SQLite / ClickHouse)"]
        EventBus --> DashboardAPI["Dashboard API (Express)"]
    end

    subgraph "User Interface"
        DashboardAPI --> WebDash["Next.js Dashboard"]
        EventStore --> CLIEngine["CLI Replay Engine"]
    end
```

Technology Stack

Component	Responsibility	Technology
Logger (Agent)	Low-latency traffic interception & proxying	Rust, Tokio, Hyper
Backend (API)	Query layer, event processing, and management	Node.js, Express, TypeScript
Frontend	Real-time traffic visualization dashboard	Next.js, TailwindCSS, WebSockets
Storage	Highly indexed event storage	JSON/SQLite (Local), ClickHouse (Prod)

Getting Started

1. Prerequisite Setup

Ensure you have the following installed:

- [Rust](#) (edition 2021)
- [Node.js](#) (v18+)
- [npm](#) or [yarn](#)

2. Component Installation

Logger (Rust Proxy)

```
cd logger
cargo build --release
```

Backend (Express API)

```
cd backend
npm install
cp .env.example .env # Update environment variables if necessary
npm run dev
```

Frontend (Next.js Dashboard)

```
cd frontend
npm install
npm run dev
```

Sub-System Deep Dives

Plugin Engine

Extend DevTrace by implementing the `Plugin` trait in Rust:

```
trait Plugin {  
    fn process(&self, event: &TraceEvent);  
}  
  
// Example: Latency Analyzer, Error Detector, Auth Validator
```

Replay Engine

Replay past requests directly from the CLI to fix bugs deterministically:

```
devtrace replay --route /api/login --id <event-id>
```

Project Roadmap

- ☒ **Phase 1:** Core Request/Response interception & basic proxy logic.
- ☐ **Phase 2:** Latency tracking, structured logging, and parallel plugin execution.
- ☐ **Phase 3:** Multi-node ingestion via Kafka and persistent Event Store implementation.
- ☐ **Phase 4:** Full Replay Engine CLI and Webhook integration.
- ☐ **Phase 5:** Advanced Analytics Dashboard with Next.js & real-time WebSockets.

License

This project is for internal developer observability. All rights reserved.

Created by **@pd241008**